

RisingSI

BioForge-Harbor: Automating the Construction of Life-Science Agent Environments from Published Literature

RisingSI

https://github.com/possibleme2026-lang/rsibio-paper2env

Abstract

We present BioForge-Harbor, a multi-agent pipeline that automatically transforms published life-science research papers into executable evaluation environments standardized on the Harbor framework, and a four-stage protocol for rigorously validating their effectiveness. The bottleneck for training and evaluating self-improving AI agents in the life sciences is not the availability of papers—Europe PMC alone indexes over 37 million publications—but the cost of converting them into verifiable, executable tasks. RSI-Forge demonstrated that a three-agent pipeline (construction, reproduction, review) can turn papers into executable environments at scale, producing 210 environments across 18 fields, 90 of which were reviewed by human experts [*]. Harbor provides the containerized evaluation infrastructure [1], with existing life-science task sets such as DrugDiscoveryBench (82 Harbor-formatted tasks) [*] and Terminal-Bench-Science [*]. The missing piece is an automated bridge between them. BioForge-Harbor closes that gap. We propose a four-stage validation protocol—*environment quality verification* → *weight-free self-improvement* → *harness optimization* → *post-training with held-out benchmarks*—that aligns with RSI-Forge's evidence chain while adding the causal evidence needed to claim that generated environments improve model capabilities, not merely provide improvement signals.

We additionally release the pipeline itself as an open-source framework. Because the empirical stages are expensive and not yet executed, we separate sharply between what is *implemented and verified* (the pipeline, its four-gate verifier, and the protocol implementation; claims C1–C8 in Appendix B are measured and each names the test that measures it) and what is *anticipated* (all ten empirical claims P1–P10 are registered as `planned`). No number in this manuscript is presented as a measurement unless a script in the released repository produces it.

Keywords: agent environments, bioinformatics, Harbor, self-improvement, benchmark construction, reinforcement learning with verifiable rewards.

Contents

1 Introduction **2**

1.1 The bottleneck is not papers. It is environments. 2
1.2 HARBOR provides the execution substrate. 3
1.3 Contribution 3

2 Related Work **4**
2.1 Automated environment construction from papers 4
2.2 HARBOR and containerized agent evaluation 4
2.3 Life-science agent benchmarks 4
2.4 Reinforcement learning with verifiable and rubric-based rewards 4
2.5 Held-out benchmarks and contamination 5

3 Method: the BioForge-Harbor Pipeline **5**
3.1 Design principles 5
3.2 Pipeline architecture 5
3.3 HARBOR task format 6
3.4 Subfield prioritization 6
3.5 Safety 6

4 Four-Stage Validation Protocol **7**
4.1 Stage 1: environment quality verification (no weight updates) 7
4.2 Stage 2: weight-free self-improvement (no weight updates) 7
4.3 Stage 3: harness optimization (no weight updates) 8
4.4 Stage 4: post-training with held-out benchmarks (weight updates) 8

5 Expected Results and Contributions **9**
5.1 Contributions 9
5.2 Anticipated findings 9

6 The Released Framework: Verification and Reproducibility **10**
6.1 Four verification gates 10
6.2 Resumability and provenance 10
6.3 Offline reproducibility 10
6.4 Limitations of the artifact 11

7 Risks and Mitigations **11**

8 Conclusion **11**

A Open Questions **12**

B Claims Ledger **12**

1 Introduction

1.1 The bottleneck is not papers. It is environments.

Life-science research produces an unprecedented volume of computationally specified methods. Europe PMC offers programmatic access to over 37 million publications, including 3.2 million fully

open-access articles [2]. PubMed Central, bioRxiv, and medRxiv provide additional APIs for bulk retrieval. The raw material for constructing agent environments is abundant and openly accessible.

The bottleneck lies in the *transformation from paper to executable environment*. An executable environment requires not only a code implementation of the method but also a reliable evaluator that distinguishes genuine methodological contributions from shortcuts. A benchmark study of biomedical paper reproduction (ClaroAI-Bench) found that *environment reconstruction is the dimension with the highest disagreement among evaluators*, exceeding even code availability as a source of variance [*]. Papers with accessible data and code achieve reproducibility scores 2.9× higher than those with restricted access, but a substantial fraction of published work does not meet this threshold.

1.2 Harbor provides the execution substrate.

The HARBOR framework, developed by Laude Institute (creators of Terminal-Bench), provides a standardized substrate for agent evaluation in containerized environments [1]. HARBOR is the official harness for Terminal-Bench 2.0 [*] and supports arbitrary agents including Claude Code and open-source models. In the life sciences, HARBOR-compatible task sets already exist:

- **DrugDiscoveryBench** (Scale AI): 82 HARBOR-formatted tasks spanning target identification, patent mining, cheminformatics, and structural reasoning, each graded by an LLM judge against expert-authored rubrics [*].
- **Terminal-Bench-Science**: 70 expert-curated research workflow tasks covering life sciences, physics, and earth sciences [*].
- **BioAgent Gym**: a biomedical agent repository with a native HARBOR leaderboard [*].
- **RSI-Exam**: evaluation infrastructure that runs directly on HARBOR for assessing long-horizon self-improvement and generalization to unseen data [*].

These task sets are *manually curated by domain experts*. The missing component is automation: a pipeline that can generate HARBOR tasks directly from the published literature.

1.3 Contribution

We propose BIOFORGE-HARBOR, comprising:

1. A **multi-agent pipeline** that ingests open-access life-science papers and outputs HARBOR-formatted executable environments with automated evaluators.
2. A **four-stage validation protocol** that progresses from environment quality verification through weight-free self-improvement and harness optimization to post-training with held-out benchmarks.
3. A **scoped rollout plan** focused on computational biology and bioinformatics, the subfields where papers are most readily convertible to executable tasks.

A fourth contribution is pragmatic but load-bearing: we release the pipeline as a dependency-free open-source framework in which every environment must pass four verification gates before it can be packaged (§6). The framework's structural claims are measured by its own test suite (Appendix B); its empirical claims are explicitly registered as not-yet-measured.

2 Related Work

2.1 Automated environment construction from papers

RSI-Forge introduced a multi-agent pipeline that turns published papers into executable environments for self-improvement [*]. Three agents coordinate *construction, reproduction, and review*: a construction agent generates code and task definitions; a reproduction agent independently reimplements the method to establish a baseline score; and a review agent checks task validity and evaluator reliability. The pipeline produced 210 environments across 18 fields, including 90 reviewed by independent human domain experts.

In a key experiment, four models performed three sequential attempts across 120 environments, inheriting code and notes but *without weight updates*. At least one model improved after the first attempt in **84% of environments**, and in **68 of 120** environments model performance exceeded the reproduced baseline—demonstrating that these environments provide genuine improvement signals. BioForge-Harbor adopts the same three roles and adds two: a *Curator* that screens the literature before any generation cost is incurred, and a *Packager* that promotes validated environments into the standardized Harbor layout (§3).

2.2 Harbor and containerized agent evaluation

Harbor is a framework from the creators of Terminal-Bench for evaluating and optimizing agents and language models in container environments [1]. Terminal-Bench 2.0 emphasizes *task validation quality* to ensure reliable and reproducible agent evaluation [*]. Harbor has been adopted by external benchmarks; for example, CompileBench migrated to Harbor to standardize agent evaluation for compiling open-source projects [*].

2.3 Life-science agent benchmarks

DrugDiscoveryBench measures how AI agents handle computational tasks in early-stage drug discovery, with 82 verifiable, domain-expert-curated tasks spanning target identification to structure–activity analysis [*]. BixBench provides over 50 real-world scenarios and nearly 300 open-ended questions in computational biology, with model accuracies as low as 17% in complex bioinformatics workflows [*]. ScienceAgentBench includes 102 scientific data-analysis tasks across computational biology, geoscience, psychology, and materials science [*]. LAB-Bench provides a further set of literature-scale biological reasoning tasks [*]. All of these are hand-built; none is generated from the literature automatically.

2.4 Reinforcement learning with verifiable and rubric-based rewards

RLVR (Reinforcement Learning with Verifiable Rewards) has emerged as a paradigm for equipping LLMs with agentic capabilities through verifiable reward signals [*]. Recent work extends RLVR through group verification-based policy optimization (GVPO) [*] and entropy-aware training [*]. **Rubrics as Rewards (RaR)** addresses the limitation of RLVR in domains without a single standard answer by using structured, checklist-style rubrics as interpretable reward signals [*]. RaR achieves relative improvements of up to 31% on HealthBench and 7% on GPQA-Diamond over LLM-as-judge baselines. Stage 4 of our protocol (§4.4) is designed to use both regimes, because life-science tasks split cleanly into those with a verifiable metric (AUROC, F1, RMSE, peak-calling accuracy) and those requiring a rubric.

2.5 Held-out benchmarks and contamination

Benchmark contamination—where test data appears in training corpora—can inflate performance estimates by as much as 10 percentage points [*]. Truly held-out evaluation requires data the model has *definitively never encountered*, often approximated through time-based splits or newly released examination questions. Our Stage 4 therefore trains only on environments derived from papers published before 2024 and evaluates on environments derived from 2025–2026 papers, *locked before training begins* and audited for n-gram overlap (§4.4, claim P10).

3 Method: the BioForge-Harbor Pipeline

3.1 Design principles

1. **Focus on computable subfields.** Prioritize computational biology and bioinformatics, where papers describe algorithms, models, or analysis pipelines with standardized inputs and outputs.

2. **Automation-first, expert-review-as-backstop.** The multi-agent pipeline generates environments; human experts review a subset for fidelity and shortcut resistance.

3. **Standardized output.** All environments are packaged as HARBOR tasks supporting containerized execution, automated scoring, and result storage.

4. **Verification before packaging.** No environment is published unless it passes the four gates of §6.1. A missing dependency or an absent grader reports UNVERIFIED, never a pass.

3.2 Pipeline architecture

> Literature Acquisition → Paper Parsing → Environment Construction → Baseline Reproduction → Quality Review → HARBOR Packaging → Evaluation & Iteration

Curator Agent (new). Screens Europe PMC, PMC, and bioRxiv for papers meeting computability criteria: open-access full text, computational biology/bioinformatics classification, available code or data, and defined evaluation metrics (F1, AUROC, RMSE, etc.). Target: 1,000 papers screened, 200 selected. In the released framework the screening rules are deterministic (licence, publication year, abstract length, subfield keyword); a model is consulted *only* for boundary papers that pass every rule except the computational classification, so the selection is reproducible and auditable.

Construction Agent. Generates the Dockerfile, dependency installation, task description, input data, and evaluation script, producing an initial executable environment. Its output is schema-checked against a strict blueprint schema before anything is written to disk.

Reproduction Agent. Independently reimplements the paper's method to establish a baseline score through a *second channel* that does not see the construction agent's own claimed numbers; agreement between the two channels—not either channel alone—is the signal. If the environment fails to run, an automated debugging loop attempts resolution; persistent failures are flagged for expert intervention.

Review Agent. Checks fidelity to the source paper, resistance to shortcut strategies, baseline stability, and safety. Biosafety screening is *deterministic and fail-closed*: a dual-use denylist is applied to the task objective before any model is consulted, and a match blocks packaging outright (claim C3). An ambiguous or erroring review is recorded as a finding, never as a silent pass.

Packager Agent (new). Converts validated environments into HARBOR task directories, re-verifying at the last gate and writing a machine-readable manifest line per environment—the artifact that Stage 1 tooling consumes.

Human Expert. Reviews a curated subset of environments, focusing on methodological fidelity and evaluator validity.

3.3 Harbor task format

We target HARBOR 0.24's on-disk layout, which the released framework cross-validates against directly (claim C2):

```
tasks/<slug>/
  task.toml               # Harbor TaskConfig, schema_version = "1.4"
  instruction.md          # the task as the agent sees it
  environment/Dockerfile  # pre-configured bioinformatics toolchain
  tests/test.sh           # agent execution entrypoint -> grader
  tests/grade.py          # automated scorer (F1 / AUROC / RMSE / ...)
  solution/               # reference solution
  rubric.md               # expert-authored rubric (optional)
```

Why the format matters for verification. HARBOR's `TaskConfig` model ignores unknown keys by default, so a mistyped resource key (e.g. `cpu` where `cpus` is meant) is *silently dropped* and the task runs with unintended limits. Our first verification gate is therefore an embedded JSON Schema with `additionalProperties: false` at every level, which rejects such a key outright (claim C1).

3.4 Subfield prioritization

Initial rollout focuses on subfields with high standardization and clear evaluation metrics: single-cell RNA-seq analysis; variant annotation and pathogenicity prediction; protein structure prediction; phylogenetic analysis; biological image segmentation; and gene regulatory network inference.

3.5 Safety

All generated tasks are restricted to *computational analysis*. Dual-use objectives (gain-of-function, pathogen enhancement, and related terms) are rejected by a deterministic denylist applied before any model call, and any hit blocks packaging (claim C3). This is a floor, not a ceiling: it catches the objective stated in the task text, and expert review remains the backstop.

[0]Earlier descriptions of HARBOR task sets used a `task.yaml` + `inputs/` + `run.sh` + `evaluator/` layout. We follow the format that HARBOR 0.24 actually validates, and our emitter is checked against the upstream `TaskConfig` model rather than against that earlier description.

Table 1: Stage 1 metrics. "Direction" is stated explicitly because two of these read in opposite senses: a higher shortcut *success* rate is worse. The symbol under each metric is its name in `validation/metrics.py`, so any reported number can be checked against the implementation.

Metric	Definition	Dir.
Construction success rate `build_success_rate`	Fraction of environments that build and run without error	↑
Baseline reproduction consistency `baseline_reproduction_consistency`	Deviation from paper-reported results	↑
Expert approval rate `expert_approval_rate`	Fraction of environments approved by human reviewers	↑
Shortcut success rate[1] `shortcut_success_rate`	Fraction of environments where the evaluator is deceived by a trivial strategy	↓

4 Four-Stage Validation Protocol

The protocol is ordered by the strength of evidence it yields: an environment that merely runs (Stage 1) is not yet evidence that it teaches anything (Stage 2), which is not yet evidence about *what* is being optimized (Stage 3), which is not yet evidence of causal capability gain (Stage 4). Each stage is implemented in the released framework and produces one machine-readable report.

4.1 Stage 1: environment quality verification (no weight updates)

Goal. Verify that generated environments are executable, reproducible, and faithful to their source papers.

Every Stage 1 metric returns `None`—not 0.0—on empty input, so an empty set can never be transcribed as a score of zero (claim C5).

Controls. Expert-manually constructed environments (DrugDiscoveryBench [*], Terminal-Bench-Science [*]) serve as the gold standard; claims P1–P4 name the control arm each rate is measured against.

Execution modes. The runner consumes a per-environment results manifest (offline scoring), can build and run each container directly when Docker is present, or merges both (disk inventory for existence, external facts for execution). When Docker is absent the affected environments are reported UNVERIFIED; they are counted, never dropped and never counted as successes.

4.2 Stage 2: weight-free self-improvement (no weight updates)

Goal. Verify that environments provide genuine improvement signals across sequential attempts.

[1]The draft nomenclature calls this "shortcut detection rate" while defining it as the fraction where the evaluator *is deceived*—a defect rate that decreases as quality improves. We keep the definition and rename the quantity to match its direction; the implementation reports both `shortcut_success_rate` (↓) and `shortcut_detection_rate` = 1 − success (↑) so either convention can be read from the report without re-deriving it (claim C7).

Protocol. Following RSI-Forge's design, N models perform K sequential attempts per environment, inheriting code and notes but *without weight updates*. We measure the first-attempt improvement rate (claim P5), the fraction of environments where some model exceeds the reproduced baseline (claim P6), and the continuous improvement rate. The scheduler is deterministic—environment-major, then round, then model—so a run is reproducible, and it supports a dry-run mode that emits the full execution plan without invoking any model.

Expected outcome. Replication of RSI-Forge's finding of $\geq 84\%$ first-attempt improvement in life-science environments [*].

4.3 Stage 3: harness optimization (no weight updates)

Goal. Verify that agents can improve through optimization of external components—prompts, tools, memory, control flow—without weight changes.

Protocol. Evaluate on HarnessOpt-Bench-style tasks [*] measuring whether models can self-modify their execution harness, and compare harness variants across OfficeQA [*], GAIA [*], and life-science HARBOR tasks. The key finding to test is whether model differences exceed harness differences, as HarnessOpt-Bench reports, or whether the opposite holds in life-science domains where domain toolchains (Bioconductor, samtools) may dominate (claim P7).

Implementation note. The comparison decomposes observed spread into between-model and between-harness components on a model $\times$ harness grid. This decomposition is only interpretable on a *complete rectangular* grid: with missing cells the two components acquire different constant offsets and "which axis dominates" becomes an artifact of which cells happen to be filled. We therefore require a complete grid before reporting the comparison; this is recorded as a known limitation in Appendix 6.4.

4.4 Stage 4: post-training with held-out benchmarks (weight updates)

Goal. Produce causal evidence that generated environments improve model capabilities on unseen tasks.

Training data. Automatically generated HARBOR environments from papers published **before 2024**.

Held-out benchmark. Environments generated from papers published **2025–2026**, *locked before training begins*. The lock records the corpus digest, the split parameters, and the membership of every arm; a later change to the corpus fails verification instead of silently drifting to match whatever the training set ended up containing (claim C6). Supplement with expert-manual tasks (DrugDiscoveryBench, Terminal-Bench-Science) and public scientific benchmarks (BixBench [*], LAB-Bench [*], ScienceAgentBench [*]) after confirming no contamination.

Training methods.

- **RLVR**: for tasks with verifiable metrics (AUROC, F1, RMSE, peak-calling accuracy) [*, *, *].
- **RaR (Rubrics as Rewards)**: for composite or subjective tasks where no single standard answer exists [*].

Table 2: Stage 4 control arms. Every comparative claim names the arm it is measured against (Appendix B); a claim without a control is not reportable in this manuscript.

Control	Purpose
No environment training	Baseline capability
Random environment training	Environment signal vs. generic training
Expert environment training	Upper bound from manual curation
Auto environment training	Effect of automation (P8)
Auto + expert-reviewed training	Effect of expert review (P9)

- **SFT / rejection sampling**: generate trajectories with a strong model and distill to the target model.

Evaluation benchmarks. Life-science agent benchmarks (BixBench, LAB-Bench, ScienceAgentBench, DrugDiscoveryBench, Terminal-Bench-Science, RSI-Exam, GAIA); self-built held-out HARBOR environments from 2025–2026 papers; and real end-to-end workflows (single-cell analysis, variant annotation, protein structure prediction).

Statistical rigor. Multiple random seeds; confidence intervals; compute-matched controls; ablations over environment count, domain diversity, evaluator strictness, and expert review proportion; and a contamination check confirming that benchmark tasks do not appear in training papers or environments (claim P10).

5 Expected Results and Contributions

5.1 Contributions

1. **First automated pipeline** from life-science papers to HARBOR-formatted executable environments.
2. **Open-source environment set**: 100+ executable, verifiable HARBOR tasks spanning computational biology subfields.
3. **Causal evaluation**: evidence of whether automatically generated environments improve model capabilities on held-out scientific benchmarks.
4. **Generalizable methodology**: a pipeline architecture extensible to physics, chemistry, and earth sciences.
5. **Community infrastructure**: complementary to DrugDiscoveryBench, Terminal-Bench-Science, and BioAgent Gym.

5.2 Anticipated findings

The following are *expectations*, not measurements. Each is registered in Appendix B with status `planned` together with the control arm it must be measured against; none of them is reported as a result anywhere in this manuscript.

- **Stage 1**: construction success rate of 60–80% for computational biology papers with available code; expert approval rate of 70–85% for the reviewed subset (P1, P3).
- **Stage 2**: replication of RSI-Forge's ≥84% first-attempt improvement rate in life-science environments (P5).
- **Stage 3**: model differences exceeding harness differences, consistent with HarnessOpt-Bench [*], unless domain-specific toolchains (e.g. Bioconductor, samtools) dominate (P7).
- **Stage 4**: statistically significant improvement on held-out benchmarks for RLVR and RaR training compared to no-environment controls, with the strongest gains in tasks matching the training subfields and partial transfer to cross-subfield tasks (P8, P9).

6 The Released Framework: Verification and Reproducibility

The pipeline is released as a dependency-free Python package (standard library only; Python ≥ 3.12). Its purpose in this paper is not to report results but to make the protocol *executable and checkable*: every stage below is runnable, and every structural claim in Appendix B names the test that establishes it.

6.1 Four verification gates

Every emitted task directory must pass all four gates before it can be packaged.

1. **Strict schema.** An embedded JSON Schema with `additionalProperties: false` at every level. This is what catches HARBOR's silent key-drop (claim C1).
2. **Layout sanity.** The files `harbor run` actually needs, in the places it expects them.
3. **Upstream cross-check.** Optional: the task directory is loaded with the real HARBOR models when HARBOR is installed. When it is absent the gate reports UNVERIFIED—a third state, distinct from pass and fail, so an environment can never be credited as verified on a machine that could not check it (claim C2).
4. **Hygiene.** LF line endings and no path escaping the task directory. A stray CRLF turns a `tests/test.sh` into a syntax error inside a Linux container, and nothing upstream reports that clearly.

6.2 Resumability and provenance

Stage progress is persisted per paper after every stage, so a crashed run resumes and a resumed run issues *zero* model calls (claim C4). Each packaged environment carries its source paper's identifier and licence in `task.toml` provenance, and the set-level manifest records the metric and verification status of every environment.

6.3 Offline reproducibility

The full pipeline and all four protocol stages have offline paths: a replay backend (canned responses), a dry-run backend (schema-valid synthetic output), and manifest-driven scoring for Stage 1. Consequently the entire surface is exercised in continuous integration without a model, a container runtime, or network access (claim C8), and a reviewer can reproduce any run deterministically.

Table 3: Risks and the mitigation built into the pipeline or protocol.

Risk	Mitigation
Data/code unavailability	Prioritize open-access papers; degrade to synthetic data or partial tasks
Complex environment dependencies	Container snapshots; Conda/Spack resolution; LLM-driven debugging loops
Evaluator shortcuts	Multi-evaluator voting; adversarial testing; expert review; shortcut probes reported in both directions (claim C7)
Biosafety concerns	Computational analysis only; prohibited tasks; a deterministic denylist applied before any model call (claim C3)
High compute cost	Cached images; parallel sandboxes; subfield-priority rollout
Limited expert resources	Review only critical environments; auto-generated rubrics for expert refinement
Benchmark contamination	Time-based splits; held-out papers locked before training; contamination audits (claims C6, P10)
Silent environment defects	Four verification gates with an explicit UNVERIFIED state; nothing is packaged unverified (§6.1)
Numbers outrunning evidence	Machine-readable claim ledger (Appendix B); a claim cannot be marked measured without evidence on disk

6.4 Limitations of the artifact

- No environment has been constructed from a live paper and no model has been run in the released version; all empirical claims are `planned`.
- Variance decomposition is reliable only on complete model × harness grids (§4.3).
- The n-gram contamination probe can miss a verbatim leak when a probe text is shorter than the n-gram window, because a short probe's single gram cannot match an 8-gram of the training text. Reported overlap is therefore a lower bound.
- Harness deltas treat an absent key as 0.0, so "did not participate" can be read as "regressed".
- Stage 1 summaries do not yet expose the review and probe denominators, so the reviewed-only denominator of the expert approval rate is not auditable from the report alone.

These are tracked in the repository and are re-checked before any number is transcribed into this manuscript.

7 Risks and Mitigations

8 Conclusion

BIOFORGE-HARBOR addresses a concrete gap: while RSI-Forge [*] demonstrates that papers can be automatically converted into executable environments, and HARBOR [1] provides standardized infrastructure for agent evaluation with existing life-science task sets, *no public work bridges these components systematically.* We propose a pipeline and a four-stage validation protocol that progresses from environment quality verification through weight-free self-improvement and harness

optimization to post-training with held-out benchmarks. The protocol aligns with RSI-Forge's existing evidence chain while adding the causal evidence—post-training effects on unseen tasks—required to claim that generated environments improve model capabilities, not merely provide improvement signals. The initial rollout focuses on computational biology and bioinformatics, the subfields where automation is most feasible and impact is most immediate.

Because the empirical stages are expensive, we have separated the claim space explicitly: the pipeline, its verification gates, and the protocol implementation are released and measured (Appendix B), while every empirical number remains registered as *planned* with the control arm it will be measured against. The ledger is the contract with the reader: no number becomes a result until a script in the repository produces it.

Author contributions. Conceptualization, methodology, software, writing—original draft preparation.

Competing interests. The authors declare no competing interests.

Data availability. Pipeline code and generated HARBOR environments will be released upon publication.

A Open Questions

- How to automatically determine whether a paper is "computable"? Our current answer is a deterministic rule set with a model consulted only on boundary papers; whether that boundary decision can be made reliably remains open.
- How to design evaluators that are both faithful and shortcut-resistant? Fidelity and shortcut-resistance can pull in opposite directions: an evaluator strict enough to reject shortcuts may also reject valid alternative solutions.
- How to handle tasks requiring proprietary data or GPU clusters?
- How to balance automation scale against expert review depth? The Stage 4 control arms (P8, P9) are designed to measure exactly this trade-off rather than assume it.
- How to prevent generated environments from enabling dangerous biological research? Our denylist is a floor; it inspects the stated objective, not the reachable capability.

B Claims Ledger

Every assertion this manuscript is allowed to make is registered in `claims.json` and reproduced here mechanically. Statuses: `measured` = a script or test in the released repository establishes it (evidence paths listed); `planned` = the protocol is implemented but the experiment has not been run, so no number is reported. `tools/check_claims.py` fails the build if a claim is marked measured without evidence on disk, if a comparative or rate claim lacks a control arm, or if the manuscript references an id that is not registered here.
Summary: 18 claims (measured 8, planned 10).

C1 (measured, structural) The strict JSON Schema gate rejects a mistyped Harbor TaskConfig key (e.g. ‘cpu‘ where ‘cpus‘ is meant) instead of letting pydantic silently drop it.
Evidence: `tests/test_verifier.py`, `src/bioforge_harbor/schemas/task_toml_1_4.json`

C2 (measured, structural) The golden Harbor task fixture cross-validates against real harbor 0.24.0 when it is installed, and the gate reports UNVERIFIED rather than passing when it is absent.
Evidence: `tests/fixtures/golden_task`, `tools/harbor_real_gate.py`

C3 (measured, structural) The dual-use denylist blocks packaging of a task whose objective matches it, fail-closed, before any model is consulted.
Evidence: `tests/test_pipeline.py`, `src/bioforge_harbor/pipeline/stages/reviewer.py`

C4 (measured, structural) Resuming a completed run issues zero backend calls; no stage is silently re-executed.
Evidence: `tests/test_pipeline.py`, `src/bioforge_harbor/pipeline/runner.py`

C5 (measured, structural) Every Stage 1-4 metric returns None on empty input instead of 0.0, so an empty set can never be read as a score of zero.
Evidence: `tests/test_metrics.py`, `src/bioforge_harbor/validation/metrics.py`

C6 (measured, structural) The held-out split is bound to a corpus digest before any training; a later corpus change makes the lock fail verification instead of drifting.
Evidence: `tests/test_stage4.py`, `src/bioforge_harbor/validation/stage4_heldout.py`

C7 (measured, structural) Shortcut probing is reported in both directions (success DOWN, detection UP) so a defect rate cannot be transcribed into a table as a win.
Evidence: `tests/test_metrics.py`, `src/bioforge_harbor/validation/metrics.py`

C8 (measured, structural) Every CLI command family has an offline green path: no model, no Docker, no network.
Evidence: `tests/test_cli.py`

P1 (planned, rate) Constructed environments build and run at a rate comparable to human-authored Harbor tasks on the same paper sample (paper 4.1).
Metric: build_success_rate — *Control:* human-authored Harbor tasks built from the same paper sample

P2 (planned, rate) Baselines re-derived by the Reproduction agent agree with the numbers reported in the source papers (paper 4.1).
Metric: baseline_reproduction_consistency — *Control:* paper-reported metric as reference, tolerance declared per environment

P3 (planned, rate) Domain experts approve constructed environments at a rate above the random-paper control (paper 4.1).
Metric: expert_approval_rate — *Control:* environments built from randomly sampled off-topic papers, reviewed blind

P4 (planned, rate) Trivial shortcut strategies are caught by the constructed evaluators (paper 4.1).
Metric: shortcut_detection_rate — *Control:* same evaluators probed with the reference solution (must pass) as the positive control

P5 (planned, rate) Agents improve after the first attempt within an environment, with no weight updates (paper 4.2).
Metric: first_attempt_improvement_rate — *Control:* same model and environment, no inherited notes or code from the previous attempt

P6 (planned, rate) In a majority of environments some model exceeds the reproduced baseline (paper 4.2).
Metric: exceeded_baseline_rate — *Control:* reproduced baseline score for the same environment

P7 (planned, comparative) Between-model variance exceeds between-harness variance on the constructed grid (paper 4.3).
Metric: variance_decomposition — *Control:* the same models under the baseline harness configuration, complete rectangular grid only

P8 (planned, comparative) Environments built from auto-selected papers outperform environments built from randomly selected papers on held-out benchmarks (paper 4.4).
Metric: control_contrast — *Control:* random_environment arm and no_environment arm, identical training recipe

P9 (planned, comparative) Adding the Review agent improves held-out scores over un-reviewed auto-built environments (paper 4.4).
Metric: control_contrast — *Control:* auto_environment arm without human review, identical training recipe

P10 (planned, rate) Held-out 2025-2026 papers share no long n-gram spans with the training corpus, so Stage 4 gains are interpretable (paper 4.4).
Metric: ngram_contamination — *Control:* same probe run against pre-cutoff papers as a positive control for the detector

References

[*] RSI-Forge: a multi-agent pipeline (construction, reproduction, review) that converts published papers into executable environments for self-improvement. 210 environments across 18 fields; 90 reviewed by human domain experts. Preprint.

[1] Laude Institute. *Harbor*: a framework for evaluating and optimizing agents and language models in container environments. https://pypi.org/project/harbor/

[*] Terminal-Bench 2.0: a benchmark for agentic terminal use, with an emphasis on task validation quality.

[*] Scale AI. DrugDiscoveryBench: 82 verifiable, domain-expert-curated tasks in early-stage drug discovery, formatted for Harbor.

[*] Terminal-Bench-Science: 70 expert-curated research workflow tasks spanning life sciences, physics, and earth sciences.

[*] BioAgent Gym: a biomedical agent repository with a native Harbor leaderboard.

[*] RSI-Exam: evaluation infrastructure running on Harbor for long-horizon self-improvement and generalization to unseen data.

[*] CompileBench: agent evaluation for compiling open-source projects, migrated to Harbor.

[*] BixBench: 50+ real-world scenarios and nearly 300 open-ended questions in computational biology.

[*] ScienceAgentBench: 102 scientific data-analysis tasks across computational biology, geoscience, psychology, and materials science.

[*] LAB-Bench: a benchmark of literature-scale biological reasoning tasks.

[*] Reinforcement learning with verifiable rewards (RLVR) for equipping LLMs with agentic capabilities.

[*] Group verification-based policy optimization (GVPO).

[*] Entropy-aware reinforcement learning with verifiable rewards.

[*] Rubrics as Rewards (RaR): structured, checklist-style rubrics as interpretable reward signals where no single standard answer exists.

[*] On benchmark contamination and the inflation of performance estimates from test data present in training corpora.

[*] HarnessOpt-Bench: measuring whether models can self-modify their execution harness.

[*] OfficeQA: a benchmark for question answering over office documents.

[*] GAIA: a benchmark for general AI assistants.

[*] ClaroAI-Bench: a benchmark study of biomedical paper reproduction, finding environment reconstruction to be the dimension with highest evaluator disagreement.

[2] Europe PMC: programmatic access to over 37 million publications, including 3.2 million fully open-access articles. https://europepmc.org/RestfulWebService